

A07_CVE-2021-3116

Here is a snippet of code written in python. I want to check for any potential security vulnerabilities.

```
from typing import Optional
```

```
from ..exception import ProxyAuthenticationFailed
from ...common.flag import flags
from ...common.constants import DEFAULT_BASIC_AUTH
from ...http.parser import HttpParser
from ...http.proxy import HttpProxyBasePlugin
```

```
flags.add_argument(
    '--basic-auth',
    type=str,
    default=DEFAULT_BASIC_AUTH,
    help='Default: No authentication. Specify colon separated user:password '
    'to enable basic authentication.')
```

```
class AuthPlugin(HttpProxyBasePlugin):
    """Performs proxy authentication."""
```

```
    def before_upstream_connection(
        self, request: HttpParser) -> Optional[HttpParser]:
        if self.flags.auth_code:
            if b'proxy-authorization' not in request.headers:
                raise ProxyAuthenticationFailed()
            parts = request.headers[b'proxy-authorization']
            [1].split()
            if len(parts) != 2 \
                and parts[0].lower() != b'basic' \
                and parts[1] != self.flags.auth_code:
                raise ProxyAuthenticationFailed()
            return request
```

```
    def handle_client_request(
        self, request: HttpParser) -> Optional[HttpParser]:
```

```
return request

def handle_upstream_chunk(self, chunk: memoryview) ->
memoryview:
    return chunk

def on_upstream_connection_close(self) -> None:
    pass
```

Here's a security review of your Python code snippet to highlight any potential vulnerabilities and recommend improvements:

Potential Security Issues

1. Weak Basic Authentication Handling:

- Basic authentication transmits credentials in an easily decodable Base64 format, which is not secure on its own unless combined with HTTPS.
- Storing or handling plain-text credentials (like `user:password`) introduces security risks.

Recommendation:

- Ensure that this proxy only runs over secure connections (HTTPS).
- If possible, use stronger authentication mechanisms (e.g., OAuth2, API keys, or tokens).

2. Improper Validation Logic:

- The logic for parsing `proxy-authorization` has some potential issues:

```
python

parts = request.headers[b'proxy-authorization'][1].split()
if len(parts) != 2 \
    and parts[0].lower() != b'basic' \
    and parts[1] != self.flags.auth_code:
```

- If `proxy-authorization` is missing, it may raise an `IndexError` when trying to access `[1]`.
- The conditions in the `if` statement are combined incorrectly using `and`, which may lead to logic errors.

Recommendation:

- Modify the condition to use `or` correctly:

```
python

if len(parts) != 2 or \
    parts[0].lower() != b'basic' or \
    parts[1] != self.flags.auth_code:
```

- Use `try / except` to handle potential `IndexError` or `KeyError`.

3. Insufficient Protection Against Header Injection or Tampering:

- The header value is split directly without sanitization, which could lead to injection attacks if headers are tampered with.

Recommendation:

- Add validation to ensure the header follows the correct structure.
- Example:

```
python

try:
    auth_scheme, credentials = parts
except ValueError:
    raise ProxyAuthenticationFailed()
if auth_scheme.lower() != b'basic' or credentials != self.flags.auth_code:
    raise ProxyAuthenticationFailed()
```

4. Lack of Rate Limiting or IP Throttling:

- If this code is deployed in a real-world environment, attackers may attempt brute-force attacks on the basic authentication credentials.

Recommendation:

- Implement rate limiting to prevent brute-force attempts.

5. No Password Hashing:

- If `self.flags.auth_code` contains a plain-text password, it can be compromised if the proxy server or logs are accessed.

Recommendation:

- Store passwords securely using hashing algorithms (e.g., bcrypt, Argon2) rather than plain text.

6. Error Handling Disclosure Risk:

- Raising `ProxyAuthenticationFailed` might reveal information about the authentication process, aiding attackers.

Recommendation:

- Return generic error messages to prevent attackers from gaining insight into which condition failed. You can log the detailed message internally but expose minimal information to the user.

Improved Code Example

python

```
class AuthPlugin(HttpProxyBasePlugin):
    """Performs proxy authentication with improved security."""

    def before_upstream_connection(
        self, request: HttpParser) -> Optional[HttpParser]:
        try:
            auth_header = request.headers.get(b'proxy-authorization')
            if not auth_header:
                raise ProxyAuthenticationFailed("Missing proxy-authorization
header.")

            parts = auth_header[1].split()
            if len(parts) != 2 or parts[0].lower() != b'basic' or \
                parts[1] != self.flags.auth_code:
                raise ProxyAuthenticationFailed("Invalid credentials.")
        except (IndexError, KeyError, ValueError):
            raise ProxyAuthenticationFailed("Malformed proxy-authorization header.")

        return request
```

Additional Recommendations:

- **Logging:** Add logging to monitor failed authentication attempts, but avoid logging sensitive information like passwords.
- **Secure Transport:** Ensure the use of HTTPS to protect credentials in transit.
- **Configuration:** Use environment variables for sensitive settings, such as the authentication code, instead of hard-coding them.

By following these recommendations, the code will be more secure and resilient to attacks like brute force and header tampering.

Provide the exact part of the code along with line numbers that are vulnerable or have security issues.

Here's a detailed review of the vulnerable parts of your code with the **exact lines** and explanations for each issue:

Vulnerable Code with Line Numbers and Explanation

python

```
17: if b'proxy-authorization' not in request.headers:
18:     raise ProxyAuthenticationFailed()
19: parts = request.headers[b'proxy-authorization'][1].split()
```

- **Issue: Missing Header Handling and Risk of `IndexError` (Line 19)**
 - If the `proxy-authorization` header is missing or malformed, accessing `[1]` directly will raise a `KeyError` or `IndexError`.
 - **Solution:** Use `try/except` to handle this gracefully or check the length of the header content.

python

```
21: if len(parts) != 2 \
22:     and parts[0].lower() != b'basic' \
23:     and parts[1] != self.flags.auth_code:
24:     raise ProxyAuthenticationFailed()
```

- **Issue: Incorrect Logic in the `if` Condition (Line 21)**

- The use of `and` instead of `or` creates faulty logic. If any part of the validation fails, it should trigger an exception.
- **Solution:** Replace `and` with `or`:

python

```
if len(parts) != 2 or \
    parts[0].lower() != b'basic' or \
    parts[1] != self.flags.auth_code:
```

python

```
17: if b'proxy-authorization' not in request.headers:
```

- **Issue: Inadequate Input Validation and Parsing (Line 17)**

- Simply checking if the header exists isn't enough. The value could still be malformed or empty.
- **Solution:** Ensure the header is correctly formatted before attempting to parse it.

python

```
7: flags.add_argument(
8:     '--basic-auth',
9:     type=str,
10:    default=DEFAULT_BASIC_AUTH,
```

```
11:     help='Default: No authentication. Specify colon separated user:password '  
12:     'to enable basic authentication.')
```

- **Issue: Storing Credentials in Plain Text (Lines 7–12)**

- The `--basic-auth` argument may expose sensitive credentials in logs, command-line history, or process lists.
- **Solution:** Avoid storing plain-text passwords directly. Use environment variables or a secure password store instead. If passwords must be passed, **hash them** using secure algorithms like `bcrypt` or `Argon2`.

```
python
```

```
19: parts = request.headers[b'proxy-authorization'][1].split()
```

- **Issue: Header Injection or Tampering (Line 19)**

- If the header content is modified maliciously (e.g., extra spaces or special characters), the code may behave unexpectedly.
- **Solution:** Validate the structure of the authorization header and use exception handling.

```
python
```

```
18: raise ProxyAuthenticationFailed()
```

- **Issue: Potential Information Disclosure (Line 18)**

- If exceptions expose details about which validation step failed (e.g., “Invalid Credentials”), it could assist attackers in guessing credentials.
- **Solution:** Raise a generic authentication error to avoid leaking sensitive information.

Summary of Vulnerable Lines

- **Line 7–12:** Credentials are exposed as plain text through arguments.
- **Line 17:** Inadequate validation for the `proxy-authorization` header.
- **Line 19:** Risk of `IndexError` and header tampering without proper checks.
- **Line 21–23:** Faulty logic using `and` instead of `or` in validation.
- **Line 18:** Potential information leakage via exception messages.

Improved Code with Fixed Vulnerabilities

Here's how you can fix these issues:

python

```
class AuthPlugin(HttpProxyBasePlugin):
    """Performs proxy authentication with improved security."""

    def before_upstream_connection(
        self, request: HttpParser) -> Optional[HttpParser]:
        try:
            # Safely retrieve and split the authorization header
            auth_header = request.headers.get(b'proxy-authorization')
            if not auth_header:
                raise ProxyAuthenticationFailed("Authentication required.")

            parts = auth_header[1].split()
            if len(parts) != 2 or \
                parts[0].lower() != b'basic' or \
                parts[1] != self.flags.auth_code:
                raise ProxyAuthenticationFailed("Invalid credentials.")
        except (IndexError, KeyError, ValueError):
            # Catch potential errors and raise a generic exception
            raise ProxyAuthenticationFailed("Invalid authentication header.")

        return request
```

By addressing these vulnerabilities, the code becomes more secure, preventing header injection, brute force attempts, and information leaks.

Please analyze the code for common security vulnerabilities if present and map them to OWASP Top 10 Web Application Security Risk category/categories.

Here's a thorough security analysis of your code, mapped to relevant **OWASP Top 10 Web Application Security Risks (2021)**:

Vulnerabilities Found and OWASP Top 10 Mapping

1. Plain-Text Authentication Handling

Code:

```
python

7: flags.add_argument(
8:     '--basic-auth',
9:     type=str,
10:    default=DEFAULT_BASIC_AUTH,
11:    help='Default: No authentication. Specify colon separated user:password '
12:    'to enable basic authentication.')
```

- **Risk:** Storing and processing authentication credentials in **plain text**.
 - **OWASP Mapping:**
 - **A2:2021 – Cryptographic Failures:** Storing and transmitting sensitive information (e.g., passwords) in plain text without encryption can lead to unauthorized access.
 - **Solution:** Avoid hardcoding credentials or storing them in plaintext. Use hashed passwords (e.g., `bcrypt`) and environment variables for configuration.
-

2. Weak Authentication Scheme (Basic Authentication)

Code:

python

```
21: if len(parts) != 2 or \
22:     parts[0].lower() != b'basic' or \
23:     parts[1] != self.flags.auth_code:
24:     raise ProxyAuthenticationFailed()
```

- **Risk:** Basic Authentication transmits credentials as Base64-encoded strings, which are easily decoded. Without TLS, these credentials are susceptible to interception.
 - **OWASP Mapping:**
 - **A7:2021 – Identification and Authentication Failures:** Using weak or insufficient authentication mechanisms can expose applications to unauthorized access.
 - **Solution:** Enforce HTTPS for all communications to secure credentials in transit. Consider stronger authentication methods (e.g., OAuth2 or tokens).
-

3. Improper Error Handling (Information Disclosure)

Code:

python

```
18: raise ProxyAuthenticationFailed()
```

- **Risk:** If detailed error messages are exposed (e.g., "Invalid Credentials"), attackers may use this information to fine-tune brute-force attacks.
- **OWASP Mapping:**
 - **A5:2021 – Security Misconfiguration:** Exposing too much information through error messages gives attackers clues about the system's inner workings.
- **Solution:** Use generic error messages in responses, such as "Authentication failed." Log detailed errors internally for debugging.

4. Inadequate Input Validation (Header Parsing)

Code:

```
python
```

```
19: parts = request.headers[b'proxy-authorization'][1].split()
```

- **Risk:** The code assumes the `proxy-authorization` header is properly structured. If malformed headers are received, the program could crash with an `IndexError` or `KeyError`.
- **OWASP Mapping:**
 - **A1:2021 – Broken Access Control:** If input (e.g., headers) isn't properly validated, attackers can manipulate requests to bypass security checks.
 - **A4:2021 – Insecure Design:** Assuming input will always follow a specific structure without validation is a design flaw that can lead to security vulnerabilities.
- **Solution:** Use proper input validation with `try/except` blocks to catch parsing errors gracefully.

5. No Rate Limiting or Brute-Force Prevention

Code: There is no specific rate-limiting mechanism in the code to prevent repeated login attempts.

- **Risk:** An attacker can attempt brute-force attacks to guess credentials, especially given the weakness of basic authentication.
- **OWASP Mapping:**
 - **A7:2021 – Identification and Authentication Failures:** Lack of rate-limiting on authentication endpoints increases the risk of brute-force attacks.
- **Solution:** Implement rate limiting, IP throttling, or CAPTCHA to prevent automated brute-force attacks.

6. Sensitive Data Exposure via Logging or Command-Line Arguments

Code:

```
python

7: flags.add_argument(
    '--basic-auth',
    type=str,
    default=DEFAULT_BASIC_AUTH,
    help='Default: No authentication. Specify colon separated user:password '
    'to enable basic authentication.')
```

- **Risk:** Command-line arguments like `--basic-auth` may be exposed in logs or process lists (e.g., `ps` output), potentially leaking sensitive data.
- **OWASP Mapping:**
 - **A2:2021 – Cryptographic Failures:** Credentials in command-line arguments may be exposed accidentally, which compromises data confidentiality.
- **Solution:** Use environment variables for sensitive data and avoid logging them directly.

7. Insecure Transport Layer (Potential Use of Plain HTTP)

- **Risk:** If the proxy does not enforce HTTPS, Basic Authentication credentials can be intercepted over the network.
- **OWASP Mapping:**
 - **A2:2021 – Cryptographic Failures:** Lack of encryption in transit exposes credentials to interception.
- **Solution:** Ensure that all communication happens over HTTPS.

Summary of OWASP Top 10 Risks Mapped to Your Code

OWASP Risk	Issue in Code	Description
A1:2021 – Broken Access Control	Improper input validation	Headers aren't properly validated, leading to potential bypass.
A2:2021 – Cryptographic Failures	Plain-text passwords and insecure transport	Storing credentials in plaintext and transmitting them without encryption.
A4:2021 – Insecure Design	Assumptions about input structure	The header parsing assumes a correct structure without validation.
A5:2021 – Security Misconfiguration	Detailed error messages	Information leakage through detailed error messages.
A7:2021 – Identification and Authentication Failures	Weak authentication and lack of rate limiting	Basic Authentication is weak and vulnerable to brute-force attacks.

Recommendations Summary

1. **Use HTTPS:** Ensure all communication between the client and proxy server uses TLS to protect credentials in transit.
2. **Avoid Plain-Text Credentials:** Use environment variables and hashed passwords instead of storing credentials in plain text.
3. **Improve Input Validation:** Validate headers thoroughly to avoid parsing errors or bypass attacks.
4. **Rate Limiting:** Implement rate limiting to prevent brute-force attacks.
5. **Error Handling:** Use generic error messages for external users and log detailed errors internally.
6. **Stronger Authentication:** Consider moving from Basic Authentication to OAuth2 or token-based authentication.

By addressing these issues, your code will be better protected against common security risks aligned with the **OWASP Top 10** guidelines.

For each identified vulnerability, provide an exploit example that an attacker could use. Include a step-by-step explanation of how the exploit works, the necessary conditions or setup, any potential limitations or obstacles

an attacker might face, and the likelihood of this attack happening. Ensure the exploits are specific to the vulnerability type provided.

Here's a detailed breakdown of the vulnerabilities identified in your code, accompanied by exploit examples, necessary conditions, limitations, and the likelihood of exploitation.

1. Plain-Text Authentication Handling

Exploit Example

- **Scenario:** An attacker monitors network traffic (e.g., using a packet sniffer like Wireshark) on an unsecured HTTP connection where credentials are transmitted in plain text using Basic Authentication.
 - **Steps:**
 1. The victim accesses a service via HTTP and submits credentials (`username:password`) as Base64-encoded text in the `Proxy-Authorization` header.
 2. The attacker intercepts the HTTP request using a tool like Wireshark or tcpdump.
 3. The attacker decodes the Base64-encoded string to reveal the plain-text username and password.
 - **Conditions:**
 - The service must be running over HTTP or an improperly configured HTTPS setup.
 - The attacker must be able to intercept network traffic (e.g., via a man-in-the-middle (MITM) attack or by being on the same network).
 - **Limitations:**
 - HTTPS encryption would mitigate this attack.
 - The attacker needs network access near the victim.
 - **Likelihood: Moderate to High** if HTTPS is not enforced.
-

2. Weak Authentication Scheme (Basic Authentication)

Exploit Example

- **Scenario:** An attacker launches a brute-force attack to guess credentials by repeatedly trying combinations of usernames and passwords.
 - **Steps:**
 1. The attacker crafts HTTP requests to the proxy server, each with a `Proxy-Authorization` header containing a Base64-encoded `username:password` pair.
 2. They use a tool like **Hydra** or a custom script to automate the attack.
 3. After many attempts, a correct credential combination is found, granting unauthorized access.
 - **Conditions:**
 - The service must accept Basic Authentication and not implement rate limiting or lockout mechanisms.
 - The attacker must know the endpoint requiring authentication.
 - **Limitations:**
 - Strong password policies could slow down brute-forcing.
 - Rate limiting or account lockouts would mitigate this.
 - **Likelihood:** **High** if rate limiting is absent.
-

3. Improper Error Handling (Information Disclosure)

Exploit Example

- **Scenario:** An attacker attempts to authenticate using incorrect or malformed credentials to identify how the system handles errors.
- **Steps:**
 1. The attacker sends HTTP requests with incorrect `Proxy-Authorization` headers (e.g., wrong username or password, invalid Base64 string).

2. The proxy server responds with detailed error messages, such as “Invalid credentials” or “Missing proxy-authorization header.”
 3. The attacker uses these responses to infer the correct structure of headers and authentication method.
- **Conditions:**
 - The application must expose detailed error messages to the attacker.
 - **Limitations:**
 - Generic error messages would prevent detailed insights.
 - Logging detailed information server-side but not exposing it externally would mitigate this.
 - **Likelihood: Moderate**, as many systems expose detailed errors by default.
-

4. Inadequate Input Validation (Header Parsing)

Exploit Example

- **Scenario:** An attacker crafts malformed `Proxy-Authorization` headers to bypass authentication or crash the proxy.
- **Steps:**
 1. The attacker sends a request with a malformed header, e.g., `Proxy-Authorization: Basic invalid_base64_string`.
 2. The server attempts to decode or parse the header, potentially raising an `IndexError` or `KeyError`, leading to a crash or inconsistent behavior.
 3. If the error is not handled gracefully, the attacker can cause denial of service (DoS).
- **Conditions:**
 - The application must not validate input properly or handle parsing errors.
- **Limitations:**
 - Proper error handling and input validation would mitigate this.
 - An attacker needs to identify endpoints processing headers.

- **Likelihood: Moderate**, as input validation issues are common.
-

5. No Rate Limiting or Brute-Force Prevention

Exploit Example

- **Scenario:** An attacker brute-forces credentials without facing any account lockouts or rate limiting.
 - **Steps:**
 1. The attacker uses a tool like **Burp Suite Intruder** or **Hydra** to automate credential guessing attempts.
 2. Requests are sent continuously without any delay.
 3. Eventually, the attacker guesses the correct credentials through sheer volume.
 - **Conditions:**
 - The service must lack rate limiting or throttling mechanisms.
 - The attacker must know the endpoint requiring authentication.
 - **Limitations:**
 - Strong passwords would slow down brute-force attacks.
 - Implementing rate limiting or CAPTCHA would mitigate this.
 - **Likelihood: High**, as brute-force attacks are common against weakly protected endpoints.
-

6. Sensitive Data Exposure via Logging or Command-Line Arguments

Exploit Example

- **Scenario:** An attacker accesses sensitive credentials through process lists or logs.
- **Steps:**

1. The attacker gains access to the server hosting the proxy application (e.g., via SSH, malware, or exploiting another vulnerability).
 2. They run a command like `ps aux | grep python` to view active processes and their arguments.
 3. The `--basic-auth` argument reveals credentials in plain text.
- **Conditions:**
 - The application must store credentials in command-line arguments or logs.
 - The attacker must have access to the server.
 - **Limitations:**
 - Limiting server access and using environment variables would mitigate this.
 - **Likelihood: Low to Moderate**, as it requires server access.
-

7. Insecure Transport Layer (HTTP Instead of HTTPS)

Exploit Example

- **Scenario:** An attacker intercepts HTTP traffic to capture credentials.
- **Steps:**
 1. The attacker sets up a MITM attack using a tool like **Ettercap** or **Bettercap**.
 2. They capture network traffic between the client and the proxy server.
 3. The attacker extracts the `Proxy-Authorization` header from intercepted requests.
 4. Base64-decoding reveals plain-text credentials.
- **Conditions:**
 - The proxy service must allow HTTP or improperly configured HTTPS.
 - The attacker must be able to intercept traffic.
- **Limitations:**
 - Enforcing HTTPS would prevent this attack.
 - TLS pinning on clients would mitigate MITM attacks.

- **Likelihood: High** if HTTPS is not enforced.

Summary of Exploit Scenarios

Vulnerability	Attack Type	Likelihood
Plain-Text Authentication	Network sniffing (MITM)	Moderate to High
Weak Authentication	Brute-force attacks	High
Improper Error Handling	Information leakage	Moderate
Inadequate Input Validation	Malformed input DoS	Moderate
No Rate Limiting	Credential brute-forcing	High
Sensitive Data in Logs/Arguments	Process inspection	Low to Moderate
Insecure Transport Layer	MITM interception	High

General Mitigation Recommendations

1. **Enforce HTTPS:** Use TLS to secure communication and prevent MITM attacks.
2. **Implement Rate Limiting:** Limit repeated login attempts to prevent brute-force attacks.
3. **Validate Inputs:** Ensure headers and inputs are properly validated and sanitized.
4. **Use Secure Authentication:** Replace Basic Authentication with stronger mechanisms (e.g., OAuth2, tokens).
5. **Secure Secrets:** Store sensitive data like credentials in environment variables and avoid exposing them in logs or process lists.
6. **Error Handling:** Use generic error messages for external responses and log details securely for debugging.

By addressing these vulnerabilities, the application will be better protected against the outlined attacks.